ПРОФЕСИОНАЛНА ГИМНАЗИЯ ПО КОМПЮТЪРНИ НАУКИ И МАТЕМАТИЧЕСКИ АНАЛИЗИ **„ПРОФ. МИНКО БАЛКАНСКИ“**

ДИПЛОМНА РАБОТА

за Държавен изпит за придобиване на трета степен на професионална квалификация

ТЕМА:

Симулатор за загуба на пакети и мрежови грешки

Ученик: Александър Валентинов Петров, 12Б клас

Професия: код 481 „Компютърни науки“

Специалност: код 4810201 „Системно програмиране“

Ръководител консултант: Ивайло Граматиков

Source Code: https://github.com/Alexandros-Digitalos/Alex-School-Project-2026

град Стара Загора

2026 година

Съдържание

УВОД 4

ГЛАВА 1: УВОД В КОМПЮТЪРНИТЕ МРЕЖИ И ПРЕДАВАНЕТО НА ДАННИ 7

1.1. Основни понятия в компютърните мрежи и комуникацията 7

1.2. Референтен модел OSI и стек от протоколи TCP/IP 8

1.3. Транспортен слой: Анализ и сравнение на TCP и UDP 11

1.4. Мрежов слой: IP протокол (IPv4 спрямо IPv6) и ролята на ICMP протокола при диагностика 14

ГЛАВА 2: КАЧЕСТВО НА ОБСЛУЖВАНЕ (QoS – QUALITY OF SERVICE) В КОМПЮТЪРНИТЕ МРЕЖИ 16

2.1. Същност и необходимост от QoS в съвременните мрежи 16

2.2. Основни метрики за измерване на качеството на мрежата 17

2.3. Влияние на мрежовия трафик върху различни видове приложения 19

2.4. Специфики на приложенията в реално време (Нееластичен трафик) 20

ГЛАВА 3: МРЕЖОВИ АНОМАЛИИ И ДЕГРАДАЦИЯ НА ВРЪЗКАТА 22

3.1. Загуба на пакети (Packet Loss) 22

3.2. Мрежово закъснение (Latency / Delay) 23

3.3. Вариация в закъснението (Jitter) 25

3.4. Ограничаване на пропускателната способност (Bandwidth Throttling / Bottlenecks) 26

ГЛАВА 4: ИНСТРУМЕНТИ ЗА ПРИХВАЩАНЕ И СИМУЛАЦИЯ НА МРЕЖОВ ТРАФИК 28

4.1. Разлика между мрежова симулация и мрежова емулация 28

4.2. Преглед на съществуващите решения 29

4.3. Технологии за филтриране и прихващане на пакети 30

4.4. Добавена стойност и приложимост на разработения проект 31

ГЛАВА 5: ОБОСНОВКА НА ИЗПОЛЗВАНИТЕ ТЕХНОЛОГИИ В ПРОЕКТА 33

5.1. Python като логически бекенд за мрежова обработка 33

5.2. Godot Engine като платформа за не-гейм UI софтуер 34

5.3. Механизъм за междупроцесна комуникация (IPC) 35

ЗАКЛЮЧЕНИЕ 37

ИЗПОЛЗВАНА ЛИТЕРАТУРА 38

ПРИЛОЖЕНИЯ 39
Godot 4.6.2 – User Interface 39
Python 3 – Backend and logic handling 39

УВОД

В ерата на повсеместната дигитализация, глобалните комуникации и облачните изчисления, компютърните мрежи представляват критичната кровеносна система, върху която функционира съвременното информационно общество. С главоломното нарастване на скоростите на трансфер и навлизането на технологии като 5G и оптични мрежи (FTTH), потребителските очаквания към софтуерните системи са по-високи от всякога. Съвременните приложения – от корпоративни платформи за видеоконферентни връзки и дистанционна телемедицина до масови мултиплейър онлайн игри и платформи за видео стрийминг – изискват не просто интернет връзка, а връзка с изключително високо качество.

Актуалност на проблема:

Основният проблем пред софтуерните инженери днес не е как да накарат приложението да работи в идеална (локална) среда, а как да гарантират неговата стабилност, когато то бъде внедрено в реалната, непредсказуема глобална мрежа. В реалния свят пакетите с данни се губят поради претоварени рутери, закъсняват заради физическата отдалеченост на сървърите и пристигат навреме, но в грешен ред. Този феномен, известен като деградация на Качеството на обслужване (Quality of Service - QoS), води до сривове в приложенията, накъсване на аудиото и неприятно преживяване за крайния потребител.

В тази връзка, наличието на надеждни софтуерни инструменти (мрежови емулатори), които могат изкуствено и контролирано да пресъздават тези негативни мрежови условия, е от критично значение за тестването и валидирането на софтуер (Stress and Resilience Testing). Именно тук се позиционира настоящата дипломна разработка.

Цел на дипломната работа:

Основната цел на проекта е проектирането, разработването и внедряването на функционален софтуерен инструмент (мрежов симулатор/емулатор) за операционната система Windows. Този инструмент трябва да бъде способен да прихваща реално-времеви мрежов трафик на ниво ядро (Kernel-level) и чрез прилагане на математически модели да деградира връзката, симулирайки загуба на пакети, високо закъснение (latency) и ограничение на пропускателната способност (bandwidth throttling). Системата цели да предложи алтернатива на съществуващите инструменти, комбинирайки мощно мрежово филтриране с изключително бърз и модерен графичен потребителски интерфейс.

Основни задачи:

За постигането на поставената цел бяха формулирани и изпълнени следните ключови задачи:

1. Теоретично изследване: Проучване на фундаментите на компютърните мрежи, моделите OSI и TCP/IP, и метриките, определящи качеството на мрежовата връзка (QoS, Jitter, Delay).
2. Анализ на технологиите: Изследване на съществуващите методи за прихващане на мрежови пакети и избор на подходяща архитектура (WinDivert).
3. Софтуерно проектиране (Бекенд): Разработка на мрежова услуга на езика Python, която да взаимодейства в реално време с мрежовия драйвер и да манипулира пакетите безопасно на ниво потребителско пространство (User Mode).
4. Разработка на Графичен интерфейс (Фронтенд): Използване на възможностите на гейм-енджина Godot за изграждане на реактивен, хардуерно-ускорен потребителски панел с възможности за визуализация на натоварването чрез съвременни графики.

5. Интеграция: Изграждане на стабилен мост за Междупроцесна комуникация (IPC) между Godot и Python чрез локални UDP сокети за предаване на команди (флагове за филтриране) в реално време без прекъсване на симулацията.
6. Тестване и валидация: Провеждане на вътрешни тестове с помощта на мрежови инструменти (Ping) за доказване на коректната работа на алгоритмите за закъснение и дроп на пакети върху даден IP адрес или порт.

Обект и предмет на изследването:

Обектът на разработката са компютърните мрежи и софтуерните инструменти за гарантиране на качеството на софтуера (QA Tools). Предметът е конкретната архитектурна и програмна реализация на хибридна система за манипулация на мрежови пакети чрез Python, WinDivert и Godot.

Настоящият документ е разделен на няколко основни части. В технологичната и теоретичната част са изложени базовите концепции зад мрежите и деградацията на връзката, последвани от обосновка на разработваната архитектура. В практическата част е разгледан самият програмен код, логиката зад потребителския интерфейс и ръководството за работа с готовия продукт.

ГЛАВА 1: УВОД В КОМПЮТЪРНИТЕ МРЕЖИ И ПРЕДАВАНЕТО НА ДАННИ

1.1. Основни понятия в компютърните мрежи и комуникацията

Съвременното информационно общество е неразривно свързано с бързия, сигурен и надежден обмен на данни. Основата на тази глобална комуникация се формира от компютърните мрежи. В своята същност компютърната мрежа представлява сложна съвкупност от автономни изчислителни устройства (наричани мрежови възли или хостове), които са свързани помежду си чрез комуникационни канали (медни кабели, оптични влакна или безжични преносни среди). Основната цел на тази инфраструктура е да позволи споделянето на ресурси и информация между потребителите и приложенията, независимо от тяхното физическо местоположение.

Историческото развитие на компютърните мрежи бележи началото си със създаването на ARPANET през късните 60-те години на миналия век, което поставя основите на съвременния Интернет. Фундаменталната концепция, която позволява толкова мащабна масовизация на мрежите, е преходът от „комутация на канали" (circuit switching, използвана в конвенционалната телефония) към „комутация на пакети" (packet switching).

При комутацията на пакети, вместо да се изгражда физическа или логическа запазена линия между източника и получателя за цялото време на сесията, обемните информационни масиви се разделят на по-малки блоки данни, наречени пакети или дейтаграми, в зависимост от конкретния протокол. Всеки мрежов пакет се състои от две основни части: потребителски данни (payload) и служебна информация (header). Хедърът съдържа метаданни като адрес на източника, адрес на получателя, пореден номер (sequence number), контролни суми за проверка за грешки и друга управляваща информация. Тази фрагментация и обособяване на данните позволяват на междинните мрежови устройства (рутери и комутатори) динамично да пренасочват пакетите към

тяхната цел по най-оптималния маршрут, съобразявайки се с текущото натоварване в мрежата. Този подход гарантира изключително висока ефективност при използването на пропускателната способност на преносната среда и е в основата на съвременните мрежови архитектури.

За да могат системи от различен тип, произведени от различни компании, базирани на разнообразни операционни системи и хардуерни архитектури да комуникират успешно, са необходими стриктни правила и конвенции за обмен на информация. Тези правила се наричат „мрежови протоколи". Мрежовият протокол дефинира формата, последователността и значението на съобщенията, обменяни между комуникиращите обекти, както и действията, които трябва да бъдат предприети при предаването или приемането на съобщение.

1.2. Референтен модел OSI и стек от протоколи TCP/IP

За да се опрости разбирането, проектирането и управлението на сложните мрежови процеси, архитектурата на мрежите е разделена на логически слоеве. Основният наръчник за концептуално стандартизиране на комуникационните функции е Референтният модел OSI (Open Systems Interconnection), дефиниран от Международната организация по стандартизация (ISO).

Моделът OSI разделя процеса на комуникация на седем независими, йерархично структурирани слоя:

1. Физически слой (Physical Layer): Най-ниското ниво на модела се занимава с предаването на сурови битове (нули и единици) през комуникационния канал. Този слой дефинира електрическите, механичните и процедурните спецификации

за активиране, поддържане и деактивиране на физическата връзка. Тук се определят нивата на напрежение, модулационните техники и типовете конектори.

2. Канален слой (Data Link Layer): Неговата основна задача е да трансформира несигурния физически канал в надеждна връзка без грешки за мрежовия слой. Той групира битовете в рамки (frames), добавя физически (MAC) адреси, за да идентифицира устройствата в локалната мрежа, и имплементира механизми за контрол на достъпа до средата (CSMA/CD например в Ethernet). Също така осъществява контрол на грешките чрез контролни суми (CRC/FCS).
3. Мрежов слой (Network Layer): Този слой отговаря за логическото адресиране на възлите в мрежата и за маршрутизирането на пакетите от източника до получателя през множество междинни мрежи. Най-разпространеният протокол тук е Интернет протоколът (IP). Мрежовият слой определя оптималния път (маршрут) чрез алгоритми за маршрутизация (OSPF, BGP).
4. Транспортен слой (Transport Layer): Осигурява прозрачен пренос на данни между крайните потребители, като предоставя надеждни услуги за пренос на данни на по-горните слоеве. Транспортният слой се грижи за надеждността на дадена връзка (ако се изисква), управлението на потока (flow control), сегментирането и десегментирането на съобщенията и контрола на претоварването (congestion control). Основно тук оперират протоколите TCP и UDP. Този слой е от ключово значение за настоящия проект, тъй като мрежовият симулатор оперира и манипулира предимно трафика на ниво транспортен и мрежов слой.
5. Сесиен слой (Session Layer): Управлява сесиите за връзка между различни приложения. Занимава се с установяване, поддържане и синхронизиране на диалога между комуникиращите страни.
6. Представителен слой (Presentation Layer): Отговаря за синтаксиса и семантиката на предаваната информация. Този слой се занимава с транслацията на данните между формата на приложението и мрежовия формат, както и с криптиране/декриптиране и компресия на файловете.

7. Приложен слой (Application Layer): Най-високото ниво в модела, което е най-близо до крайния потребител. Той предоставя интерфейс към различни мрежови услуги като електронна поща (SMTP, POP3, IMAP), трансфер на файлове (FTP), уеб сърфиране (HTTP/HTTPS) и услуги за резолвиране на имена (DNS). Този слой определя самите заявки към приложенията, които произвеждат или консумират трафик.

Докато моделът OSI е изключително важен от теоретична гледна точка, реалната световна мрежа Интернет функционира благодарение на практическата имплементация, известна като TCP/IP стек от протоколи. За разлика от седемте слоя на OSI, архитектурата на TCP/IP се състои от четири слоя:

- Слой за мрежов достъп (Network Access Layer) – обединява Физическия и Каналния слой от OSI.
- Интернет слой (Internet Layer) – съответстващ на Мрежовия слой.
- Транспортен слой (Transport Layer) – еквивалентен на Транспортния слой в OSI.
- Приложен слой (Application Layer) – обединява Приложния, Представителния и Сесийния слой от OSI.

Процесът на предаване на данни в тази йерархия е известен като капсулация (encapsulation). Докато данните пътуват надолу по стека от Приложния към Физическия слой на изпращача, всеки слой добавя свой собствен хедър (а в някои случаи инкапсулираща опашка). При получателя процесът е обратен – де-капсулация (decapsulation), при която всеки слой премахва и обработва своя специфичен хедър, преди да подаде данните към по-горното ниво. Това разделение на абстракции позволява мрежовият софтуер да се проектира модулно, позволявайки например разработването на софтуер за симулиране на мрежови грешки при транспорта или мрежовия слой, без това да нарушава базовите принципи на работа на приложенията, генериращи трафика.

1.3. Транспортен слой: Анализ и сравнение на TCP и UDP

Когато се разработва инструмент за симулация на мрежови аномалии като закъснение, загуба на пакети и jitter, е абсолютно критично да се разбира дълбоко работата на протоколите от Транспортния слой. Двата основни стълба на транспортния слой в Интернет днес продължават да бъдат TCP (Transmission Control Protocol) и UDP (User Datagram Protocol). Въпреки че и двата протокола използват IP слоя за доставка на пакетите между мрежовките интерфейси, начинът, по който те управляват крайните потоци от данни, е коренно различен. Техните характеристики предопределят изцяло как приложният софтуер ще се държи при деградирана мрежова среда.

Transmission Control Protocol (TCP)

TCP е ориентиран към свързване (connection-oriented) протокол, който предоставя надеждна доставка на байтове в правилна последователност и с гаранция за целостта им. За да се предадат данни през TCP, комуникиращите системи трябва първо да установят логическа връзка чрез процес, известен като „тройно ръкостискане" (Three-way Handshake: SYN → SYN-ACK → ACK).

Основните механизми в TCP, които осигуряват неговата надеждност и управление на трафика, включват:

- Гарантирана доставка чрез потвърждения (Acknowledgments - ACK): Всеки сегмент данни, изпратен през TCP, трябва да бъде потвърден от получателя. Ако подателят не получи потвърждение (ACK) в рамките на определен времеви прозорец (Retransmission Timeout - RTO), той приема, че пакетът е загубен в преносната среда и автоматично го изпраща отново. Това означава, че когато

мрежовият симулатор наложи изкуствена „загуба на пакети", TCP веднага ще реагира, като задейства процедурите си за ретранслация.

- Управление на последователността (Sequence control): TCP присвоява уникален Sequence Number на всеки байт който изпраща. Получателят използва тези номера, за да пренареди получените сегменти в правилната им последователност, в случай че те пристигнат разбъркани. Това свойство прави TCP имунизиран срещу разбъркване на пакетите по трасето.
- Управление на потока (Flow Control): Протоколът използва механизъм, наречен „плъзгащ се прозорец" (sliding window), който предотвратява ситуация, при която бърз подател „претоварва" със заявки по-бавен получател, информирайки колко свободен буфер има приемащата страна във всеки един момент.
- Контрол на претоварването (Congestion Control): Вероятно най-важната характеристика на TCP в контекста на мрежовите симулации е реакцията му при претоварване в мрежата. Чрез специфични алгоритми като AIMD (Additive Increase / Multiplicative Decrease), Slow Start, Congestion Avoidance и Fast Recovery, TCP постоянно следи за евентуални загуби на пакети или увеличаване на времето за пътуване (RTT - Round Trip Time). Ако детектира проблем, TCP протоколът драматично намалява скоростта на изпращане, с цел да „отпуши" мрежата. Именно поради този механизъм симулираните закъснения или дропнати пакети оказват диспропорционално голям ефект върху производителността на файлови трансфери и уеб сесии, базирани на TCP.

User Datagram Protocol (UDP)

За разлика от TCP, UDP е свързващо независим (connectionless) и ненадежден протокол. UDP осигурява механизъм за изпращане на капсулирани IP дейтаграми без да установява формална сесия с получателя. При UDP липсва процедура по установяване на връзка, липсва процес на потвърждаване за получени данни, както и механизми за

реорганизиране на объркани пакети, управление на потока или контрол против претоварване на мрежата.

Въпреки негативното звучене на термина „ненадежден", UDP има огромни предимства. Премахването на сложната служебна информация (header-ът на UDP е само 8 байта в сравнение с минимум 20 байта при TCP) и липсата на изчаквания за потвърждения правят UDP изключително бърз, ефективен и ниско-латентен протокол. UDP се изпраща „fire and forget".

Именно тези негови качества го правят единствения логичен избор за приложения, чувствителни към закъснение, където скоростта на предаване е по-важна от абсолютната цялост на данните:

- Онлайн видео игри и мултиплейър: При съвременните FPS или MOBA игри положението на курсора или играча се обновява по 60 и повече пъти в секунда. Ако пакет със стари координати се изгуби, по-добре е той да бъде игнориран и играта да продължи с най-новите данни, отколкото цялата комуникация да спре в изчакване на препредаване (както би наложил TCP). Управлението на мрежовия трафик при игрите показва ясна техническа потребност от правилно разбиране на загубите на пакети (packet loss) и вариацията им във времето (jitter).
- Voice over IP (VoIP) и видеоконферентни връзки: При загуба на част от аудио фрейм през UDP, потребителят чува леко припукване или микронакъсване в аудиото, което е човешки усвоимо и често незабележимо. Ако се ползваше TCP, гласовото предаване би спирало постоянно, трупайки забавяне от секунди заради ретранслациите, превръщайки разговора в изключително трудна и изнервяща задача.

Мрежовият симулатор има за задача да може да манипулира прецизно различни видове трафик, позволявайки по-добро изследване и тестване на софтуерни приложения в стресови условия, като диференцирането на поведението между TCP и UDP е ключова характеристика за валидността на проведените тестове.

1.4. Мрежов слой: IP протокол (IPv4 спрямо IPv6) и ролята на ICMP протокола при диагностика

Интернет протоколът (IP) е основният комуникационен протокол от Мрежовия слой (Layer 3), като неговата задача е да адресира хостовете и да маршрутизира данните от източник към получател.

IPv4 (Internet Protocol version 4) все още е най-широко разпространеният протокол. Неговата система за адресиране разчита на 32-битови адреси (често изписвани като четири десетични октета, например 192.168.1.1). Това позволява приблизително 4.3 милиарда уникални адреса, които заради масовата експанзия на технологиите са на практика изчерпани. Въпреки техники като NAT (Network Address Translation), които временно облекчават този проблем чрез споделяне на един публичен адрес между всички устройства в локална мрежа, генералното решение е преходът към IPv6.

IPv6 използва 128-битови адреси и въвежда редица подобрения като вграден IPSec, премахване на фрагментацията по трасето в междинните маршрутизатори (премествайки отговорността във възлите) и опростен хедър за по-бърза обработка. Разработваните мрежови филтри често трябва да предлагат съвместимост и с двата стандарта или да ги обработват индивидуално.

Особено важен в контекста на мрежовия анализ и симулация е протоколът съпътстващ IP – ICMP (Internet Control Message Protocol). ICMP не е предназначен за пренос на потребителски данни между приложения. Неговата роля е изцяло служебна –

той служи като диагностичен и информационен инструмент за мрежовите устройства на ниво Layer 3. Когато рутер трябва да изхвърли пакет (например поради липса на път към дестинацията или надвишено време за живот – Time Exceeded), той генерира ICMP съобщение обратно към първоначалния изпращач, информирайки го за проблема.

ICMP е механизмът, зад който стоят двата най-популярни инструмента за тестване и профилиране на мрежова свързаност:

- ping: Изпраща ICMP Echo Request съобщения до даден хост и измерва времето, което отнема да получи обратно ICMP Echo Reply. Това е основен инструмент за измерване на закъснението (Round Trip Time, RTT) и за откриване на наличието на загуба на пакети (packet loss) в една мрежа. Когато софтуерът за симулация „дропи" пакети, диагностичният ефект най-директно се наблюдава чрез инструменти като ping.
- traceroute (или tracert в Windows среди): Използва полето TTL (Time-To-Live) в IP хедъра на ICMP или UDP пакети, за да открие кои точно междинни рутери преминава трафикът.

Разбирането на взаимодействието между транспортните (TCP/UDP) и диагностичните (ICMP) протоколи позволява изграждането на софтуерни системи, които не просто нарушават свързаността на сляпо, а интелигентно модифицират мрежовите параметри, позволявайки разработчика да симулира прецизно контролирани стрес тестове. Тези параметри, техните ефекти и същността на „деградацията" на връзката са обект на следващите глави, разглеждащи по-задълбочено качеството на обслужване в компютърните мрежи.

ГЛАВА 2: КАЧЕСТВО НА ОБСЛУЖВАНЕ (QoS – QUALITY OF SERVICE) В КОМПЮТЪРНИТЕ МРЕЖИ

2.1. Същност и необходимост от QoS в съвременните мрежи

Традиционната архитектура на Интернет, базирана на протокола IP (Internet Protocol), е проектирана да функционира въз основа на парадигмата „максимално усилие" (best-effort delivery). При този модел мрежата се опитва да достави всички пакети от източника до получателя колкото се може по-бързо, но без да предоставя каквито и да е гаранции за това дали пакетът действително ще пристигне, кога ще пристигне или в какъв ред. В среда с достатъчно свободен честотен ресурс (bandwidth), моделът на максимално усилие работи безупречно. Проблемите възникват, когато мрежовият капацитет се окаже недостатъчен за обработка на постъпващия трафик – ситуация, известна като претоварване (congestion).

С еволюцията на Интернет мрежите претърпяха фундаментална трансформация. Докато първоначално те се използваха предимно за асинхронен пренос на данни (като електронна поща и трансфер на текстови файлове), днес компютърните мрежи представляват конвергирана медия. През едни и същи комуникационни канали и маршрутизатори се предават едновременно уеб трафик, поточно локално видео с висока резолюция, гласови разговори в реално време (VoIP), данни от IoT (Интернет на нещата) сензори и мултиплейър гейминг сесии.

Тази конвергенция поражда критична необходимост от механизми за диференциация и приоритизация на различните потоци данни. Именно тук се явява концепцията за Качество на обслужване (Quality of Service - QoS). QoS представлява съвкупност от технологии, механизми и архитектурни рамки, които позволяват на мрежовия администратор да управлява трафика и да гарантира определено ниво на производителност за конкретни приложения или потребители. Основната цел на QoS не е да „създаде" повече честотна лента, а да разпредели интелигентно и справедливо

наличния ограничен ресурс, като гарантира, че критичният трафик ще бъде обслужен с предимство пред по-маловажния или по-търпимия към закъснения.

Правилното разбиране на концепциите зад QoS е фундаментално при разработката на мрежов симулатор/емулатор. Инструментът, предмет на тази разработка, на практика извършва обратния процес – вместо да гарантира качеството чрез приоритизация, той умишлено деградира (влошава) метриките на обслужване по контролиран начин чрез налагане на закъснения, филтриране и отхвърляне на пакети, за да се провери устойчивостта (resilience) на софтуерните приложения в условията на „лош" QoS.

2.2. Основни метрики за измерване на качеството на мрежата

За да може качеството на една мрежова връзка да бъде дефинирано, оценено и управлявано (или симулирано като нарушено), се използват набор от количествени метрики. Всяко мрежово приложение има различни изисквания и толеранси спрямо тези параметри.

Основните параметри на QoS включват:

1. Пропускателна способност (Bandwidth / Throughput): Bandwidth (честотна лента или капацитет) дефинира теоретичния максимум от данни, които могат да бъдат пренесени по даден канал за единица време (измерва се в битове в секунда – bps, Kbps, Mbps, Gbps). Throughput (полезна пропускателна способност) е реалното количество потребителски данни, което успешно се предава през мрежата за единица време в конкретен момент. То е винаги по-малко от пълния капацитет поради служебната информация (overhead) в протоколните хедъри и евентуалните ретранслации при възникнали грешки.

2. Загуба на пакети (Packet Loss): Метрика, представляваща процента от изпратените пакети, които не успяват да достигнат до крайната си дестинация. В идеални условия този процент е 0%. Загубата може да възникне поради препълнени буфери в рутерите (когато входящият трафик надвишава скоростта на обработка), поради смущения на физическия слой (особено при безжични мрежи като Wi-Fi или 4G/5G) или зловредни действия (DDoS атаки).
3. Закъснение (Latency / Delay): Времето, необходимо на един пакет да пропътува от изпращача до получателя (One-way delay). В практиката често се измерва времето за двупосочно пътуване (Round-Trip Time - RTT), тъй като комуникацията при TCP и приложния слой разчита на потвърждения. Закъснението не е константна величина и се формира от няколко компонента: време за разпространение на сигнала по средата (propagation delay), време за сериализация на битовете в интерфейса (transmission delay), изчакване в опашки в рутерите (queueing delay) и време за преизчисляване на хедърите (processing delay).
4. Вариация на закъснението (Jitter / Packet Delay Variation - PDV): Джитер е статистическата разлика (дисперсия) във времето на пристигане на последователни пакети, принадлежащи на един и същ поток данни. Ако източникът изпраща пакети през равни интервали от 20ms, а получателят ги приема през нерегулярни интервали (напр. 15ms, 30ms, 10ms, 45ms), мрежата проявява висок Jitter. Основната причина за появата на джитер е динамично променящото се изчакване в опашките на рутерите (queueing) в зависимост от текущото натоварване от други потоци данни.

Всички тези метрики са взаимносвързани. Високото закъснение, голямата вариация (jitter) и липсата на достатъчно пропускателна способност обикновено завършват с препълване на буферите в рутерите, което директно води до загуба на пакети (packet drop), предизвиквайки така наречения каскаден спад в производителността (cascading performance degradation).

2.3. Влияние на мрежовия трафик върху различни видове приложения

Мрежовият трафик не е напълно хомогенен. За да се анализира ефективно влиянието на мрежовите грешки (или симулирани такива) върху софтуера, трафикът обикновено се класифицира в две големи категории: еластичен (elastic) и нееластичен (inelastic / real-time).

Еластичен (асинхронен) трафик

Еластичният трафик произлиза от приложения, които са изключително толерантни към закъснения в мрежата, но са абсолютно нетолерантни към нарушения в целостта на данните (загуба на информация). Тези приложения могат „еластично" да увеличават или намаляват скоростта на изпращане на данни в зависимост от наличния мрежов капацитет, адаптирайки се към условията.

- Трансфер на файлове (FTP, SMB, HTTP Downloads): Когато потребителят сваля голям файл или го изпраща по електронната поща, за него обикновено няма значение дали процесът ще отнеме 2 секунди или 5 секунди. От критична важност е обаче файлът да бъде получен без нито един липсващ или обърнат байт, иначе целият архив или документ ще бъде повреден/нечетим.
- Уеб сърфиране (HTTP/HTTPS): При зареждане на уеб страница, потребителят очаква отговор сравнително бързо, но системата толерира временни забавяния. Също поради стриктните изисквания към целостта на информацията (HTML, JavaScript кодът трябва да са 100% точни), уеб трафикът използва протокола TCP.
- Реакция на мрежови аномалии: Когато симулатор на мрежов трафик наложи висока загуба на пакети върху еластичен трафик, TCP архитектурата скрива тази загуба от самото приложение. Приложението никога няма да получи повреден файл. Вместо това, TCP ще препрати (retransmit) изгубените пакети. Крайният

резултат, наблюдаван от потребителя, ще бъде единствено спад в скоростта (throughput) – файлът, който нормално се сваля за 10 секунди, може да отнеме 3 минути при 5% симулирана загуба на пакети, тъй като TCP алгоритмите за контрол на претоварването ще интерпретират загубата като знак за задръстване в мрежата и ще „задушат" прозореца на изпращане (congestion window).

Интерактивен (Interactive) трафик

Това са приложения като SSH, Telnet или връзки за дистанционен контрол (Remote Desktop). Тези потоци и мрежови услуги генерират много нисък обем от данни (често само координати на мишка или единични натискания на клавиатура), което означава, че не изискват почти никаква пропускателна способност. При тях обаче закъснението (latency) е изключително важно. Ако пакет отнеме повече от 150-200 милисекунди, потребителят ще усети осезаем „лаг" между натискането на клавиш и изписването му на терминала. Тук също загубата на пакети е неприемлива и се задействат механизми за препредаване.

2.4. Специфики на приложенията в реално време (Нееластичен трафик)

Нееластичният трафик, често наричан реалновремеви (real-time) трафик, генерира най-сложните изисквания към съвременните IP мрежи. Това са приложения, при които стойността и полезността на информацията спада драстично или напълно изчезва, ако тя не бъде доставена в рамките на строги времеви ограничения. Тези приложения използват предимно протокола UDP и имат специфично поведение при наличие на деградирала свързаност.

- VoIP (Глас по IP) и Видеоконференции (Zoom, Teams, Discord): При двупосочна гласова или видео комуникация, човешкото възприятие е решаващият фактор. За да се поддържа естествен ритъм на разговор, общото мрежово еднопосочно

закъснение не бива да надвишава 150 милисекунди (според препоръките на ITU-T). Ако забавянето надвиши 300 ms, участниците започват несъзнателно да говорят един през друг (talk-over effect), което превръща комуникацията в изключително трудна и изнервяща. Този тип трафик толерира малка загуба на пакети (до 1-2%). Загубеният аудио фрагмент се компенсира на ниво приложен софтуер чрез алгоритми за прикриване на грешки (Packet Loss Concealment – PLC).

- Справяне с Jitter (Jitter Buffers): Може би най-големият враг на реално-времевите стрийминг технологии не е константното закъснение, а вариацията – Jitter. За да възпроизведат глас или видео гладко, софтуерните клиенти използват Jitter Buffer (Джитер буфер). Това е участък от локалната памет, в който пристигащите пакети изкуствено се задържат за кратко време преди възпроизвеждане (напр. допълнителни 40-50ms). Ако един инструмент за мрежови симулации генерира Jitter по-голям от размера на буфера, софтуерът ще изхвърли закъснелите пакети и крайният резултат ще бъде прекъсване и накъсване (stuttering) на медията.
- Онлайн Мултиплейър Игри и Cloud Gaming: Тук стандартите към качеството са най-завишени. Игрите от тип First-Person Shooter (FPS) изискват възможно най-малко закъснение (ping/latency под 50ms е идеален, до 100ms е приемлив). Всяка загуба на пакети води до феномена известен като „rubberbanding", при който клиентът и сървърът се разминават за състоянието на играча, а в последствие клиентът бива върнат няколко метра назад на екрана.

Обобщено, познаването на изискванията към QoS за различните типове трафик е фундаментална теоретична постановка при създаването на симулатор на мрежови аномалии. Това знание позволява да се предвиди как трябва да се държи определено приложение, когато софтуерът изкуствено лимитира честотната лента или започне контролирано да отхвърля пакети, което е критична стъпка при тестването на механизмите за толерантност към откази.

ГЛАВА 3: МРЕЖОВИ АНОМАЛИИ И ДЕГРАДАЦИЯ НА ВРЪЗКАТА

3.1. Загуба на пакети (Packet Loss)

В една перфектна комуникационна система всеки предаден бит информация достига своя получател мигновено и без грешки. В реалната физическа среда на глобалната мрежа обаче, това далеч не е така. Състоянието, при което един или няколко мрежови пакети, пътуващи в компютърна мрежа, не успеят да достигнат крайната си дестинация, се нарича „загуба на пакети" (Packet Loss). Тази аномалия е фундаментален проблем за мрежовото инженерство, тъй като директно нарушава целостта на предаваното съобщение.

Основни причини за възникване на загуба на пакети:

1. Претоварване на мрежовия трафик (Network Congestion): Най-често срещаната причина. Маршрутизаторите и комутаторите разполагат с ограничени по обем буфери (опашки), в които съхраняват пакетите преди препращането им към следващия възел. Когато обемът на входящия трафик през даден интерфейс системно надвишава пропускателната способност на изходящия интерфейс, тези буфери се запълват. След като буферът е пълен на 100%, мрежовото устройство няма друг избор освен просто да „отхвърли" (drop) всеки следващ постъпващ пакет. Най-простият алгоритъм за това се нарича Tail Drop.
2. Грешки при предаване във физическата среда: Повреди в кабелната инфраструктура, електромагнитни смущения, радио смущения (особено при Wi-Fi свързаност) може да доведат до „обръщане" на битове в пакета. Когато пакетът пристигне при получателя, контролната сума (например Frame Check Sequence в Ethernet) се изчислява отново и се сравнява с изпратената. Ако двете суми не съвпадат, пакетът се обявява за повреден и се изхвърля на Канален слой (Data Link Layer).

3. Неправилно конфигуриран софтуер / Хардуерни грешки: Забавена обработка от процесорите на междинните рутери или прекалено рестриктивни правила в защитните стени (Firewalls) също водят до загуби.

Методи за компенсиране на загубите: В компютърните комуникации са се наложили няколко основни стратегии за редуциране на ефекта на изгубените пакети, наричани общо методи за възстановяване на грешки.

- Автоматично препредаване (ARQ - Automatic Repeat reQuest): Този подход връща отговорността на подателя. Чрез използването на система от потвърждения (ACK) и таймери (Timeouts), изпращачът е способен да засече липсващ пакет и да го изпрати отново. Това е основата, на която работи протоколът TCP. Процесът обаче води до допълнително натоварване на мрежата и добавя значително закъснение.
- Права корекция на грешки (FEC - Forward Error Correction): Този алгоритмичен подход се използва масово при предаване на аудио и видео стрийминг с UDP, където препредаването (ARQ) би отнело прекалено много време. При FEC изпращачът добавя математически изчислена излишна (redundant) информация към полезния товар. Ако получателят регистрира загуба на част от пакета, той използва сложни математически модели на база умишлено предадения излишен код, за да интерполира или възстанови загубената информация.

3.2. Мрежово закъснение (Latency / Delay)

Закъснението дефинира интервала от време, измерван предимно в милисекунди (ms), който е необходим на една единица данни да измине разстоянието по мрежовата топология от източника до получателя.

Технически погледнато, общото мрежово закъснение не е една конкретна характеристика на линията, а се формира като сумарна сборна компонент от четири напълно отделни фактора по време на пътя на всеки пакет. Математическата формула описваща закъснението е:

Delay_total = Delay_processing + Delay_queueing + Delay_transmission + Delay_propagation

Нека разгледаме по-подробно всеки от тези компоненти:

1. Закъснение при обработка (Processing Delay): Времето, необходимо на процесора на мрежовото устройство (рутер, комутатор) да прочете хедъра на пакета, да провери битови грешки в съдържанието му, да разпознае дестинацията и да направи изчисление в маршрутната си таблица чрез кой интерфейс да бъде насочен пакетът. Провежданите тук операции обикновено се извършват от специализирани хардуерни контролери с много висока скорост (в границите на няколко микросекунди).
2. Забавяне в опашката (Queueing Delay): Най-динамичният и непредсказуем компонент на закъснението. След като пакетът бъде насочен към правилната изходна връзка от рутера, той трябва да изчака своя ред в буфера, тъй като портът обикновено може да излъчва само по един пакет едновременно към физическата медия. Ако няма други чакащи пакети, това закъснение е нула. Но при периоди на масивен мрежов трафик (bursts), този параметър може да достигне десетки и стотици милисекунди.
3. Време за предаване (Transmission Delay): Този параметър зависи директно от дължината на информационния пакет (размера му в битове, L) и пропускателната скорост на използвания конкретен канал (R - измерван в битове/сек).

Предавателното закъснение е равно на L/R. Важно е да се отбележи, че това НЕ е времето необходимо за пътуване на данните от единия възел към другия.

4. Закъснение на разпространение (Propagation Delay): След като един бит е физически подаден върху свързващия кабел (мрежов сегмент), на него му е нужно време да достигне до другия край. Това закъснение представлява съотношение между физическата отдалеченост между устройствата (d) и скоростта на разпространение на вълната на електромагнитния сигнал в дадената среда (s). Скоростта е в пряка зависимост от скоростта на светлината и в медни или оптични кабели е от порядъка на ≈ 2·10^8 метра в секунда.

3.3. Вариация в закъснението (Jitter)

Както беше споменато в предходната глава, голямо, но стабилно закъснение може да се окаже малък проблем в сравнение с малко, но силно вариращо такова. Този феномен се нарича Jitter (от английски – трептене).

Дефиницията според стандартите на ITU е: абсолютната вариация във времето на разпространение (packet delay variation) на пакетите, принадлежащи към един непрекъснат поток. По-научно, ако пакетите се генерират при източника през прецизен времеви интервал Δt, а пристигат в получателя през интервали t1, t2, …, tn, тогава джитер се явява статистическата дисперсия на поредицата времеви закъснения.

Основният източник за появата на джитер е сложната комбинация от множество информационни потоци, идващи от хиляди хостове из световната мрежа с различни скорости. Тяхната интеракция вътре в междинните комутационни буфери води до това, че отделни пакети от един разговор попадат на претъпкани опашки (увеличавайки закъснението им), а последващи пакети влизат в кратки или празни опашки

(преминавайки почти веднага), което води до събирането им времево един до друг при получателя.

При разработването на симулатор на мрежови проблеми (каквато е целта на настоящия проект) имплементацията на jitter-алгоритъм изисква навлизането в архитектури за буфериране. Симулаторът не просто задържа всички преминали пакети за 100ms, а въвежда генератор на псевдослучайни числа на база нормално (Гаусово) разпределение или Разпределение на Поасон покрай средното зададено закъснение (base delay), динамично прилагайки времеви задръжки върху различни пакети. По този начин симулацията отговаря максимално прецизно на математическия модел, описващ процесите в рутирането из реална мрежа.

3.4. Ограничаване на пропускателната способност (Bandwidth Throttling / Bottlenecks)

Ограничаването на мрежовия капацитет, познато още като ограничаване на честотната лента (bandwidth throttling или traffic shaping), е административна или физическа интервенция, при която скоростта на предаване на данни се редуцира до предварително зададен максимален лимит.

В реалния свят това често се извършва умишлено от Интернет доставчиците (ISP), с цел предотвратяване на претоварване на тяхната инфраструктура или налагане на абонаментни планове с различни скорости. Тази технология е фундаментална и за корпоративни мрежи, където служи за ограничаване скоростта на по-маловажни услуги.

Има два основни популярни архитектурни дизайна за симулиране или налагане на такова ограничение в рамките на софтуерно приложение за контрол на трафика:

- Управление чрез изхвърляне (Traffic Policing): При този метод мрежовият възел активно следи пропускателната скорост на потока от информация. При надминаването на предварително договорения лимит, възелът просто започва да превключва пакетите (извършва packet drop). Този метод създава своеобразна изкуствена загуба на пакети за „наказан" софтуер, разчитайки, че протоколът на по-горно ниво (TCP) ще реагира чрез намаляване на скоростта на своята трансмисия.
- Оформяне на трафика (Traffic Shaping): При него пакетите, които превишават наличната или разрешената пропускателна способност, не се изхвърлят веднага. Вместо това те се съхраняват във временен буфер (опашка) – механизъм, наречен Leaky Bucket (Течаща кофа) или Token Bucket (Кофа с жетони - най-често прилагания). Реализацията на мрежовия симулатор на практика включва модули, симулиращи поведението именно на тези буфериращи и филтриращи механизми.

ГЛАВА 4: ИНСТРУМЕНТИ ЗА ПРИХВАЩАНЕ И СИМУЛАЦИЯ НА МРЕЖОВ ТРАФИК

4.1. Разлика между мрежова симулация и мрежова емулация

В академичната и инженерната общност термините „симулатор" и „емулатор", когато става въпрос за компютърни мрежи, често се използват взаимозаменяемо в неформални разговори. От техническа и архитектурна гледна точка обаче има фундаментална разлика между двете концепции, която е важно да бъде изяснена в рамките на този проект.

Мрежовата симулация е процес, при който поведението на една мрежа (устройства, кабели, протоколи) се моделира изцяло математически чрез софтуер, без да се генерира, приема или предава реален мрежов трафик. Симулаторите, като популярният академичен инструмент ns-3 (Network Simulator 3), създават абстрактни виртуални възли, чието състояние се изчислява във времето въз основа на вероятностни математически модели. При симулацията понятието „време" също е виртуално – симулацията може да пресметне 10 секунди свързаност в рамките на няколко милисекунди или обратното. Главното им предимство е способността да тестват теоретични сценарии в огромни мащаби, без да е необходим специализиран хардуер.

Мрежовата емулация (или понякога наричана „симулация в реално време на физически хардуер"), от своя страна, означава, че софтуерната система взаимодейства пряко с реална, функционираща компютърна мрежа на живо. Емулаторът се явява софтуерен възел, през който преминава реален мрежов трафик, генериран от живи потребителски приложения, като браузъри или игри. Емулаторът прехваща този реално-времеви трафик и умишлено налага върху него математически модели на влошаване (забавяне, отхвърляне), след което го предава отново в мрежата.

Разработваният софтуер тук де факто е емулатор на мрежови условия, въпреки че заради наложилите се разговорни стандарти потребителите често го разпознават като „симулатор на лаг".

4.2. Преглед на съществуващите решения

На пазара и в open-source общността съществуват редица инструменти, създадени с цел емулация на деградирала мрежа, като голяма част от тях се използват от разработчици на софтуер, за да тестват устойчивостта на системите си (resilience testing).

1. NetEm (Network Emulator) / tc: NetEm е модул, който е вграден директно в ядрото (kernel) на операционната система GNU/Linux. Той е де факто стандарт за индустрията в Linux средите. В комбинация с инструмента tc (Traffic Control), той позволява на системните администратори изключително фин контрол: ограничаване на честотната лента, внасяне на jitter базиран на перфектни Гаусови разпределения, умишлено дублиране или повреждане на IP пакети. Недостатъкът е, че инструментът изисква работа от терминал (CLI) и е неприложим директно върху клиентски машини, базирани на Windows.
2. WANem (Wide Area Network Emulator): Софтуерен продукт с отворен код, изграден като bootable Linux ISO, базиран на Knoppix и NetEm. Архитектурата му е предназначена да превърне стар настолен компютър в специализиран рутер между две локални мрежи, който да емулира бавна глобална мрежа (WAN) между тях. Добавя предимството на наличието на базов графичен уеб интерфейс към NetEm.
3. Clumsy: Малък, преносим софтуер за Windows платформи, чиято единствена цел е симулиране на лоша мрежова свързаност. Инструментът не изисква инсталация и използва лек графичен потребителски интерфейс за активиране на закъснения или дропване на пакети. Техническата му архитектура (базирана на WinDivert) е

много близка до търсената в настоящата дипломна работа. Недостатъкът му се крие в ограничената възможност за фино управление по време на работа (real-time graphs) и остарелия интерфейс.

Разработваният тук проект търси да обедини мощта на програмното филтриране и гъвкавия модерен потребителски интерфейс чрез използване на специализирани архитектури (Godot и Python). За да може това да се случи обаче, основното софтуерно предизвикателство в една Windows среда остава прихващането на самите пакети.

4.3. Технологии за филтриране и прихващане на пакети

Операционните системи са проектирани да крият мрежовите детайли от потребителските програми. Когато едно приложение като Google Chrome отвори сокет и изпрати HTTP GET заявка, то не създава мрежови слоеве, IP адреси или MAC хедъри – то изпраща текст към ядрото (Network Stack в OS ядрото), а ядрото върши всичко останало. Следователно един мрежов емулатор трябва да оперира на значително по-ниско ниво, за да може да прочете или манипулира вече генерираните пакети преди те да стигнат до мрежовата карта.

Средата на Windows предоставя няколко технически решения за това:

- Pcap (WinPcap / Npcap): Това е най-известната библиотека (разработена първоначално за Unix като libpcap), на която се базира известният анализатор Wireshark. Архитектурата ѝ позволява да се копира всяка рамка (frame), която преминава през мрежовата карта. Проблемът тук е, че pcap е проектиран основно за снифинг (подслушване/анализ). Макар библиотеките да имат възможност за инжектиране на сурови пакети, те нямат механизъм за налагане на „veto" или

предотвратяване на оригиналния пакет да бъде изпратен. Поради това pcap архитектурата е неподходяща за изграждане на delay/packet loss емулатори.

- NDIS филтриращи драйвери и Windows Filtering Platform (WFP): За да се блокират и манипулират пакети, се изисква драйвер, оперираща в Ring-0 (Kernel Mode). Microsoft предоставят WFP – мощен набор от API в ядрото, който позволява прозрачно филтриране на мрежови данни. Този метод е изключително бърз и надежден, но разработката на директен кернел драйвер е процедура, характеризираща се с огромна сложност. Всяка грешка или memory leak води директно до срив на цялата операционна система (Blue Screen of Death - BSOD), а крайният драйвер задължително трябва да бъде дигитално подписан.
- WinDivert (Windows Packet Divert): Библиотеката WinDivert предоставя перфектния компромис, съчетавайки мощта на кернел мода с безопасността на потребителското пространство (User-Mode). WinDivert се състои от WFP callout драйвер (WinDivert.sys) и потребителска библиотека (WinDivert.dll). Драйверът на ниво ядро се инсталира автоматично и прихваща пакетите, отговарящи на определен филтър. Вместо да пусне тези пакети към физическата мрежова карта, драйверът ги „отклонява" (diverts) и ги блокира временно. Потребителската програма (Python скриптът) извиква WinDivert библиотеката, за да прочете задържания пакет. Програмата има пълна свобода да го анализира, модифицира, забави или изцяло да го изхвърли.

4.4. Добавена стойност и приложимост на разработения проект

Въпреки наличността на описаните съществуващи решения като Clumsy и NetEm, настоящият дипломен проект за мрежов симулатор/емулатор има своята ясна обосновка и добавена стойност.

Около 90% от персоналните компютри са базирани на Windows, което елиминира Linux-базираните kernel инструменти (като tc) за местни, бързи клиентски тестове на софтуер от Windows разработчици. От друга страна, популярните инструменти за Windows често изостават визуално и технологично, бидейки писани на остарели UI рамки без модерни функционалности като вградени графики за анализ на натоварването в реално време.

Проектът интегрира фундаментално различен дизайн:

1. Разединена архитектура: Разделянето на тежката мрежова логика за управление на пакетите от графичния интерфейс. Създава се специализиран бекенд сървис (мрежов прехващач), който е оптимизиран за производителност.Модерен интерфейс и аналитика: Вместо стандартните статични форми, използването на специализиран гейм-енджин позволява изчертаването на мащабни и плавни графики на латенцията, вариацията на пакетите и текущия честотен капацитет, създавайки много по-добра визуална обратна връзка за тестващия инженер.

2. Образователна приложимост: Ясната архитектура и възможността за контролирани стрес-тестове прави инструмента идеален не само за QA (Quality Assurance) специалисти, но и като образователен софтуер за демонстрация на ефектите на различните параметри на Транспортния слой и Качеството на обслужването в академична среда.

ГЛАВА 5: ОБОСНОВКА НА ИЗПОЛЗВАНИТЕ ТЕХНОЛОГИИ В ПРОЕКТА

Проектирането на съвременни софтуерни системи изисква внимателен подбор на технологичен стек, който да отговаря на специфичните нужди за производителност, надеждност и бързина на разработка (Rapid Application Development). За реализацията на мрежовия симулатор е избран нетрадиционен, но изключително ефективен хибриден подход – разделяне на приложението на два независими модула: системен бекенд, написан на Python, и графичен интерфейс (фронтенд), изграден чрез гейм-енджина Godot.

5.1. Python като логически бекенд за мрежова обработка

Изборът на език за програмиране, който да оперира с мрежови пакети на системно ниво, често се свежда до C или C++. Тези езици предоставят директен достъп до паметта и нулево допълнително закъснение в изпълнението на инструкциите (overhead). Въпреки това управлението на паметта (memory management) при тях е изцяло ръчно, което драстично увеличава риска от критични грешки и забавя цикъла на разработка.

Python се налага като отлична алтернатива на нисковъзходящите езици за мрежова обработка, благодарение на богатата си екосистема и механизмите си за абстракция:

1. Библиотечни обвивки (Wrappers): Python позволява безпроблемна интеграция с външни, компилирани на C библиотеки чрез интерфейси като ctypes или cffi. Така официалната системна библиотека WinDivert.dll може да бъде управлявана чрез неофициални високо-производителни Python модули като pydivert. Това осигурява скоростта на C в ядрото, съчетана с четимостта и лесната поддръжка на обектно-ориентиран код от високо ниво.

2. Асинхронно програмиране: Съвременните версии на Python предоставят вградени механизми за асинхронно и многонишково (multithreading/multiprocessing) изпълнение. Тъй като прихващането на пакети е процес, зависим предимно от входно-изходни операции (I/O Bound), отделните цикли в скрипта могат да прихващат пакета в една нишка, докато друга нишка прилага задръжките на латенцията (sleep/wait), предотвратявайки блокирането на главния поток на програмата (deadlock).
3. Обработка на данни: Езикът е безкомпромисен лидер в манипулирането на даннови структури и JSON формати, което го прави идеален за бързо парсване (parsing) на команди, получени от графичния интерфейс.

5.2. Godot Engine като платформа за не-гейм UI софтуер

Един от най-иновативните моменти в този проект е използването на среда за разработка на игри (Godot Engine) за създаването на приложен Desktop софтуер.

Традиционните рамки за разработка на потребителски интерфейси под Windows (като Windows Forms, WPF или дори екосистеми като Electron и Qt) имат редица ограничения. Qt изисква сложни среди за компилация (C++), Electron консумира огромно количество оперативна памет (RAM), тъй като де факто стартира цял уеб браузър (Chromium) на заден план, а WinForms е функционално морално остарял.

Godot Engine е проектиран за изобразяване на 2D и 3D компютърна графика при минимум 60 кадъра в секунда. Този модел на рендване, използващ хардуерно ускорение (OpenGL / Vulkan директно през GPU), предоставя изключителни предимства:

- Гъвкава възлова архитектура (Node System): В Godot всеки елемент (бутон, надпис, контейнер) се явява възел (Node) в огромно дърво. Тази строго обектно-

ориентирана и логическа структура създава интуитивен модел за сглобяване на сложен, реактивен софтуерен интерфейс без многократно писане на повтарящ се код.

- Визуализация на данни в реално време: Основната слабост на традиционните UI инструменти (като Clumsy) е липсата на графики. За да може един мрежов инженер да провери дали емулаторът наистина прилага Jitter или закъснение от 200ms, той има нужда от визуално доказателство. Godot може лесно да чертае хиляди линии и вектори всяка секунда, позволявайки създаването на плавни осцилоскопи и монитори за трафика (Dynamic LineGraphs).
- GDScript: Godot използва собствен скриптов език, наречен GDScript, чийто синтаксис е силно повлиян от Python. Това улеснява огромно работата на един разработчик (Full-stack developer), който не трябва да превключва логическото си мислене между коренно различни парадигми.
- Стилове и теми (Theming): Godot разполага с изключително мощна система за създаване на модерно изглеждащи панели и бутони чрез така наречените StyleBox обекти, поддържащи градиенти, заобляния, сенки в реално време и плавни преходи (animations/tweening) при задържане на мишката.

5.3. Механизъм за междупроцесна комуникация (IPC)

Най-голямото предизвикателство в хибридни системи, разчитащи на два отделни процеса (в случая: godot_ui.exe и python_backend.exe), е методът за тяхната синхронизация и предаване на команди. Механизмите за решаване на този проблем се категоризират като средства за Междупроцесна комуникация (IPC – Inter-Process Communication).

Съществуват различни подходи като споделена памет (Shared Memory), семафори и съобщения, през Named Pipes. Използваната топология в проекта обаче се доверява на

класическия мрежов стек (Localhost Sockets) поради неговата надеждност и възможността архитектурата лесно да се скалира в бъдеще.

За комуникационен канал е избран протоколът UDP (User Datagram Protocol) върху Loopback адреса (127.0.0.1). Изборът на UDP пред по-надеждния TCP за локална комуникация има няколко фундаментални инженерни обосновки:

4. Липса на TCP Overhead: UDP е протокол предоставящ минимално забавяне, тъй като не изисква първоначално тройно ръкостискане и управление на сесиите.
5. Надеждност на Localhost: Докато в глобалната мрежа UDP е ненадежден при изгубени пакети, в рамките на един и същ компютър операционната система рутира UDP пакетите изцяло през локалната виртуална памет. Те никога не напускат компютъра и не достигат до физическия кабел, което означава, че загуба на пакети физически не съществува върху интерфейса 127.0.0.1.
6. Еднопосочност на командите (Fire-and-forget): Графичният интерфейс (Godot) непрекъснато изпраща данни до бекенда (Python). Тези данни представляват конфигурационен низ в JSON формат, съдържащ параметрите от потребителския интерфейс, например: { "bandwidth_limit": 1000, "delay_ms": 150, "drop_rate_pct": 5.5, "target_ip": "192.168.1.5" }.

По този начин се изгражда асинхронна архитектура „Издател-Абонат" (Publisher-Subscriber) със слаба свързаност (Loose coupling). Ако графичният интерфейс крашне, бекендът ще продължи да филтрира пакети по последните подадени му правила. Ако бекендът бъде принудително затворен, Godot няма да блокира, чакайки отговор от прекъснат сокет, и ще позволи на потребителя да рестартира сървиза.

ЗАКЛЮЧЕНИЕ

Компютърните мрежи и принципите, по които те осъществяват комуникация представляват може би най-сложната и динамична технологична екосистема, създавана от човечеството. Протоколите като TCP и UDP залагат фундаментите за изграждане на устойчиви или изключително бързи връзки, но те самите са силно уязвими към аномалии в преносната среда като загуба на пакети, прекомерно натрупано закъснение, Jitter в рутерите и ограничаване на честотната лента. Растящата необходимост от Качество на обслужване (QoS) превръща управлението на тези аномалии в централен фокус както на телекомуникационната индустрия, така и на софтуерното инженерство.

Приложението, създадено в рамките на този проект, илюстрира на практика как тези теоретични концепции могат да бъдат умишлено симулирани на ниво операционна система (посредством мрежово филтриране с WinDivert). Чрез интегрирането на логическите възможности на езика Python и хардуерно ускореното графично представяне, достъпно благодарение на Godot Engine, софтуерът предлага мощен, гъвкав и интуитивен инструмент. Разработеният продукт не само отговаря на нуждата за тестване на мрежовата толерантност на софтуерни приложения в нарушена среда, но и демонстрира успешното архитектурно сътрудничество между напълно различни технологии чрез оптимално използване на протоколи за локална междупроцесна комуникация (IPC).

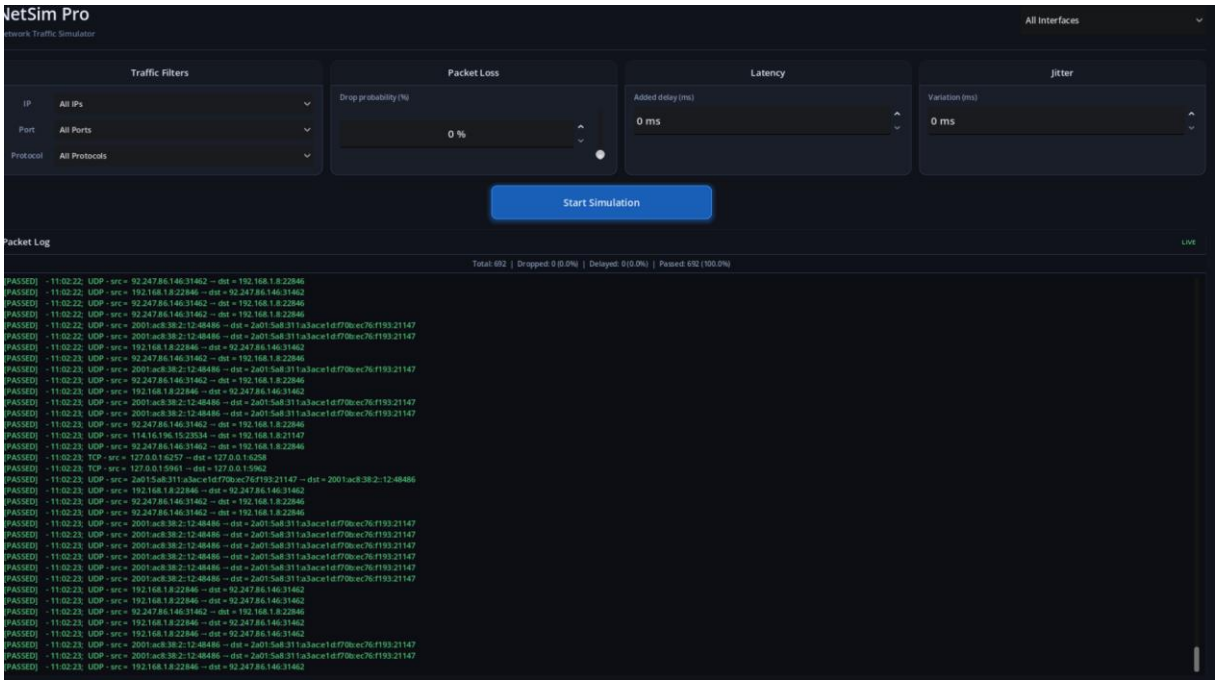
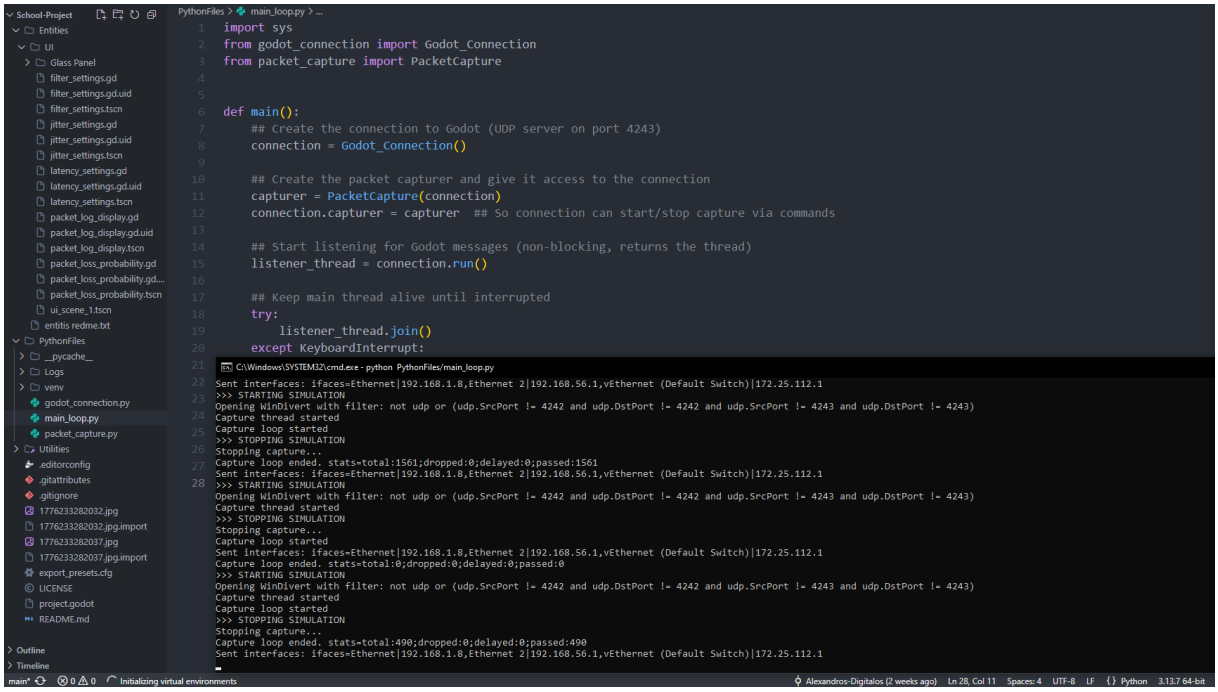
Настоящата разработка разкрива и редица пътища за бъдещо разширение на функционалността. Включването на поддръжка за IPv6 пакети, добавянето на система за запис и последващо „повторение“ (replay) на мрежови сесии под симулирани условия, или развитието на централизиран сървърен модел за дистанционен контрол на множество агенти в корпоративна лаборатория, са само малка част от потенциала, скрит в избраната архитектура. Реализираното приложение поставя солидна основа, върху която тези амбициозни цели могат методично да бъдат постигнати.

ИЗПОЛЗВАНА ЛИТЕРАТУРА

1. Tanenbaum, A. S., Wetherall, D. J. (2011). Computer Networks, 5th Edition. Prentice Hall.
2. Stevens, W. R., Fall, K. R. (2011). TCP/IP Illustrated, Volume 1: The Protocols, 2nd Edition. Addison-Wesley Professional.
3. Kurose, J. F., Ross, K. W. (2017). Computer Networking: A Top-Down Approach, 7th Edition. Pearson.
4. Forouzan, B. A. (2013). Data Communications and Networking, 5th Edition. McGraw-Hill Education.
5. Kozierok, C. M. (2005). The TCP/IP Guide: A Comprehensive, Illustrated Internet Protocols Reference. No Starch Press.
6. WinDivert Official Documentation and API Reference. https://reqrypt.org/windivert.html
7. Godot Engine Documentation. https://docs.godotengine.org
8. Python Documentation. https://docs.python.org/3/
9. RFC 793 – Transmission Control Protocol. IETF, 1981.
10. RFC 768 – User Datagram Protocol. IETF, 1980.
11. RFC 791 – Internet Protocol. IETF, 1981.
12. RFC 792 – Internet Control Message Protocol. IETF, 1981.
13. IEEE Std 802.3-2018 – Ethernet Standard. IEEE, 2018.
14. ITU-T G.114 – One-way transmission time. International Telecommunication Union, 2003.
15. Linux man-pages: tc(8), tc-netem(8). The Linux Kernel Archive.

ПРИЛОЖЕНИЯ

Godot 4.6.2 – User Interface



Python 3 – Backend and logic handling